

Article

Optimizing Apache Spark MLlib: Predictive Performance of Large-Scale Models for Big Data Analytics

Leonidas Theodorakopoulos ¹, Aristeidis Karras ^{2,*} and George A. Krimpas ²

¹ Department of Management Science and Technology, University of Patras, 26334 Patras, Greece; theodleo@upatras.gr

² Computer Engineering and Informatics Department, University of Patras, 26504 Patras, Greece; krimpas@ceid.upatras.gr

* Correspondence: akarras@ceid.upatras.gr

Abstract: In this study, we analyze the performance of the machine learning operators in Apache Spark MLlib for K-Means, Random Forest Regression, and Word2Vec. We used a multi-node Spark cluster along with collected detailed execution metrics computed from the data of diverse datasets and parameter settings. The data were used to train predictive models that had up to 98% accuracy in forecasting performance. By building actionable predictive models, our research provides a unique treatment for key hyperparameter tuning, scalability, and real-time resource allocation challenges. Specifically, the practical value of traditional models in optimizing Apache Spark MLlib workflows was shown, achieving up to 30% resource savings and a 25% reduction in processing time. These models enable system optimization, reduce the amount of computational overheads, and boost the overall performance of big data applications. Ultimately, this work not only closes significant gaps in predictive performance modeling, but also paves the way for real-time analytics over a distributed environment.

Keywords: Apache Spark; MLlib; big data processing; big data analytics; performance prediction; machine learning; resource optimization; feature engineering; emerging technologies; decision-making



Academic Editor: Ioannis Tsoulos

Received: 2 January 2025

Revised: 24 January 2025

Accepted: 28 January 2025

Published: 1 February 2025

Citation: Theodorakopoulos, L.; Karras, A.; Krimpas, G.A. Optimizing Apache Spark MLlib: Predictive Performance of Large-Scale Models for Big Data Analytics. *Algorithms* **2025**, *18*, 74. <https://doi.org/10.3390/a18020074>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Since it was first introduced, big data has changed the way structures function and extract insights from complex mass data. However, this brings two main issues, especially about the computational complexity. Apache Spark, a high-performance distributed computing environment, has emerged as one of the leading frameworks in big data computing, well received for its efficiency in data processing based on in-memory computation. This capability is expanded further yet by Spark's MLlib library which consists of a wide range of machine learning (ML) algorithms capable of analyzing big data in parallel. Nevertheless, when it comes to running MLlib algorithms in Spark, it may well prove a technically challenging and laborious process, which only gets exponentially worse with increased dataset sizes.

However, in such big data scenarios and large-scale environments, predicting the time of execution and usage of resources for running MLlib algorithms can be unpredictable [1,2]. On another level, as organizations require the possibility of real-time analytics and rapid decision-making, these computational lags are seen as essential hurdles. The complexity of enhancing the efficiency of ML workloads in Spark has therefore become one of the major areas of interest to researchers and practitioners.

Despite its massive potential as an ML library in big data processing, Apache Spark's MLlib proves to be challenging to optimize as its timings cannot be predicted and its resource usage can vary unpredictably. Although comprehensive predictive modeling for performance optimization has been generally examined in existing studies, especially for large-scale and real-time analytics, this study fills a gap by exploring a variety of aspects including multi-operator resource sharing and its relationship with dataset properties.

To address this challenge, there is a pressing need for comprehensive approaches that leverage historical performance data to forecast the computational requirements of MLlib algorithms within Apache Spark (<https://spark.apache.org/>, accessed on 2 January 2025). Existing research has primarily focused on optimizing specific aspects of Spark's performance or refining the algorithms themselves, often overlooking the potential of predictive modeling as a tool for proactive resource management. The unpredictability associated with executing MLlib algorithms on large datasets poses a substantial barrier to achieving optimal performance, leading to inefficient resource allocation and delayed analytics processes [3,4].

Motivated by this research gap, the present study aims to empower organizations with the ability to anticipate the performance of MLlib algorithms before execution. By accurately forecasting execution times and resource requirements, it becomes feasible to optimize system configurations in advance, thereby preventing computational bottlenecks and reducing resource wastage. Leveraging machine learning techniques to predict and enhance the performance of machine learning workflows introduces an innovative feedback mechanism that holds significant promise for improving the efficiency of distributed data systems [5,6].

In addition, despite the research already completed in the field to enhance the performance of Apache Spark MLlib, there are still some open issues. These include the absence of appropriate predictive models suitable for workloads of different types, resource allocation frameworks which cannot scale out, and the absence of a consideration of real-time workload features, especially in a dispersed setting. Furthermore, existing techniques are often based on static settings that cannot dynamically respond to a shift in workload. This study addresses these gaps by proposing a predictive modeling technique which uses metrics of execution in order to improve the utilization of resources and congestion in one or more processing nodes of the large-scale Spark environment.

Our work extends beyond standard approaches motivated by unresolved gaps in hyperparameter optimization, scalability in distributed systems, and the integration of external factors affecting performance. We utilize machine learning models that give us a advantages in predictive insight into MLlib operator behaviors for proactive configuration tuning. The research's necessity exists in how it may be utilized in streamlining big data applications, alleviating computational bottlenecks, and improving decisions in data analytics that require large-scale data.

To mitigate these limitations, this study presents a novel idea of how to estimate Apache Spark MLlib algorithms, given that their performance cannot be directly assessed due to their high computational complexity. Through the use of performance data, it is possible to establish performance modeling that will predict the performance metrics including the execution time of a job and the amount of resources to be used in executing it. This allows organizations to proactively tune Spark configurations to efficiently allocate the resources and eliminate computational bottlenecks. The key insight of our work is the ability of machine learning to use itself as a way to enhance the execution of other machine learning processes and form feedback loops for the betterment of distributed data systems, similar to the manner in which ML can be used on low-power devices in scenarios utilizing TinyML or Federated Learning mechanisms [7–9].

The key purpose of this study is to deliver a usable approach for increasing the efficiency in big data analytics through predictive performance modeling. We gather detailed efficiency measures from several MLlib algorithms, like K-Means Clustering, Random Forest Regression, and Word2Vec, which are executed on big, distributed datasets. Utilizing these metrics, we develop machine learning models that are able to precisely foresee algorithm performance determined by input data characteristics and system configurations. Providing a proactive alternative, these decision models can reduce execution delays and resource waste, therefore improving the big data analytics experience overall.

The remainder of this paper is structured as follows. In Section 2, the background and related work is presented, as well as existing approaches and research gaps. In Section 3, we outline the methodology used for data collection and predictive model training along with the systems and software utilized throughout the study. Section 4 evaluates the performance of our predictive models, discussing their impact on optimizing big data analytics workflows in Apache Spark along with their evaluation metrics. In Section 5, the key findings are summarized and discussed, while ultimately, our paper concludes with Sections 6 and 7, which present the Conclusions and Future Work.

2. Background and Related Work

2.1. Existing Research on Optimizing Apache Spark Performance

An analysis of prior work on Apache Spark performance enhancement identifies performance prediction, resource allocation, and machine learning algorithms as areas of interest. Research has looked at the different approaches to integrate solutions that improve the time it takes to process big data in Spark environments. Another contribution to the prediction of execution times is derived from methods that use initial execution samples to develop machine learning models that predict execution statistics like identical job time, memory usage, and I/O costs. This method was tested using the Wordcount application and showed high accuracy in predicting these metrics in Spark's standalone mode [10].

A different approach, stage-aware performance prediction, makes use of historical execution data of the Spark History Server. This technique uses Spark workload execution patterns to improve accuracy in end-to-end execution time predictions, avoiding the shortcomings of traditional holistic approaches [11]. Analytical modeling as well as hybrid approaches that combine analytical modeling with machine learning to predict performance in cloud-based big data applications are also explored. This method lowered the number of necessary experiments for accurate prediction significantly and surpassed traditional machine learning algorithms in accuracy [12]. Finally, a combination of analytical and machine learning models was used in a study of complex workflow applications to predict execution times of data-driven workflow modeled as Directed Acyclic Graphs (DAGs), demonstrating the effectiveness of hybrid models in handling unseen and complex workflows [13].

In terms of resource estimation, one study focused on optimizing memory allocation in Spark applications, where a machine learning model was developed to forecast effective memory requirements based on service level agreements (SLAs). This approach resulted in significant memory savings while maintaining acceptable execution delays [14]. Another study compared the execution times of various machine learning models, such as logistic regression and Random Forests, within the Spark MLlib framework. It was found that logistic regression had the shortest execution time, emphasizing the importance of optimization strategies like caching and persistence in Spark applications [15].

Apache Spark tuning is focused on a set of configuration properties that are sensitive and dependent on each other. An ML-based approach has been suggested to perform this tuning, which has given an improvement of 36% on average over the default value.

This method involves the use of binary and multi-classification to optimize the flow in the parameter space [16].

Other performance improvement design factors include the RDMA on high-performance networks where there have been performance improvements. Prior work shows that RDMA-based designs can improve the performance of Spark RDD operations by up to 79% and graph processing workloads by up to 46% [17]. This approach capitalizes on new network technologies such as InfiniBand; to do so, it includes an analysis of the data transferring rates.

GPU acceleration is another area of research where an ample amount of investigation is being performed and a performance gain of targeted applications as high as $17\times$ is reported, for example, in K-Means Clustering. This showcases the possibilities of post-processing with GPUs as a way of improving the computation rate of Spark [18].

The case of hyperparameters tuning Apache Spark has been implemented using reinforcement learning to provide up to 37% better speedup in comparison to the classical approaches [19]. The elements of the objectives are equally placed into consideration, such as time, memory, and accuracy, hence making this approach a perfect optimization strategy. Furthermore, enhancements to the Java-compiled code of Spark query plans have yielded factors of $1.4\times$ to improve the performance of some pointed queries and machine learning programs [20].

Ultimately, the analysis reveals that combining machine learning and analytical modeling improves performance prediction techniques in Apache Spark. Performance prediction techniques are those that utilize past performance data and machine learning models to predict important parameters such as execution time, the amount of memory, and other resource requirements for given processes or tasks. These strategies elevate prediction accuracy and minimize the inconvenience caused by lengthy experimental setups while demonstrating Spark's ability to manage a range of workloads. Maximizing resource management plays a key role in raising performance levels for big-data applications on a large scale.

Table 1 summarizes the prior research in the context of Apache Spark optimizations.

Table 1. Summary of prior research on Apache Spark optimization.

Study	Methodology	Strengths	Limitations	Advancements in This Study
[14]	SLA-based memory optimization	Significant memory savings	Limited real-time adaptability	Presents predictive modeling for dynamic resource allocation
[13]	Workflow performance prediction	Effective for DAG workflows	Inefficient for large-scale MLlib tasks	Customized for MLlib operators in distributed systems
[16]	ML-based parameter tuning	Improved average performance by 36%	Limited to static configurations	Dynamically adjusts to workload variations
[18]	GPU acceleration for Spark	Significant speed-ups for specific tasks	Not generalized for MLlib operators	Applies predictive modeling to diverse workloads

2.2. Decision-Making in Emerging Technologies for Big Data Analytics

Various sectors—marketing, healthcare, supply chain management, and manufacturing—are being remodeled through emerging technology in big data analytics-based decision-making processes. Advanced analytics tools and methodologies permit organizations to leverage gigantic datasets to extract actionable insights, thereby making better inferences.

Some research studies offer insight into how new technologies are changing access to decision-making in numerous sectors:

- **Impact of Emerging Technologies on Freight Transportation:** In this study, the authors review how technologies like artificial intelligence (AI), blockchain, and big data analytics are affecting logistics and decision-making on transportation [21–24]. Since stakeholders in logistics management would benefit from it, it analyzes nine key technologies that are currently in use and the prospects for their future.
- **Big Data Analytics for Business Growth:** Ohlhorst et al. investigate how big data analytics can be leveraged to create a competitive advantage for businesses [25]. In their book, they focus on the financial value of big data, providing tools and the best practices for implementation, and the economic importance of big data as a force behind the redefinition of markets and the improvement in profits.
- **AI and ML in Supply Chain Planning:** Vummadi et al. discuss the integration of AI, ML, and big data analytics into supply chain management in their research [26], emphasizing the need for the modernization of supply chain planning for decision-making, forecasting, and agility improvement.
- **Intelligent Big Data Analytics:** Sun et al. present a manager framework based on intelligent big data analytics for better business decision-making through planning, organizing, leading, and controlling [27].
- **Multimedia Big Data Analytics:** Pouyanfar et al. present a comprehensive view of multimedia big data analytics, considering problems and opportunities in multimedia data management and analytics on a large scale [28].
- **Big Data in Healthcare:** One study reviewed privacy and ethical considerations for big data analytics in healthcare decision-making; Sterling et al. describe the opportunities and challenges of big data in healthcare [29].
- **Perspectives on Big Data Analytics:** Ularu et al.'s article emphasizes the value of using large datasets for making strategic decisions, asserting that sometimes better algorithms do not give better results, while more data often does [30]. They further highlight the problems with processing large data volumes and their advantages [30].
- **Big Data Analytics Process Architecture:** Crowder et al. present a new data analytics architecture specifically designed to improve decision-making competence by enabling real-time data discovery and knowledge construction [31].

2.3. Performance Improvement in Distributed Frameworks

The fine-tuning of machine learning algorithms in big data platforms like Apache Spark has been another research subject in the recent past. The complexity of managing both computation and communication in distributed systems has resulted in the formulation of new frameworks and toolkits to overcome these performance issues [32–35].

CoCoNet is one of the closed frameworks that addresses the problem of the division of abstraction between computation and communication layers. CoCoNet allows for the coordination of these two dimensions such that the throughput optimizations are accrued as part of improving performance for distributed machine learning. Through the above optimization of data processing and communication, it has been proved that this framework can achieve dramatic speed-up in distributed settings [36].

Another approach, the dPRO toolkit, has been particularly designed to diagnose and improve the performance problem in distributed deep neural network (DNN) training. The bottlenecks are detected by the fine-grained profiling of applications and offer solutions to enhance training speed in detail together with the DNN training time improvement across different ML frameworks through dPRO [37].

2.4. Limitations of Current Optimization Techniques in Apache Spark MLlib

Despite these advancements in optimizing machine learning workloads, Apache Spark's MLlib still faces several limitations when it comes to performance optimization techniques:

- **Hyperparameter Optimization Challenges:** The methods in use for hyperparameter optimization (HO) usually subscribe to black box strategies, and therefore the internal processing characteristics of big data are not detected, which can lead to deceptive solutions. This leads to high computational costs, large search spaces, and vulnerability to the dimensions of the multiple objectives' functions. Moreover, to improve the HO, the reinforcement learning methods need to be applied and a few iterations are needed to update the parameters, which might take a lot of time and resources [19].
- **Resource Allocation Issues:** Apache Spark has some issues with dynamic resource allocation capabilities, which are not very efficient in handling vast variations in data traffic, for instance, in stream processing. Because the data loading is not predictable at certain times, it causes the system to fail to perform at optimal levels and hence be costly to run [38]. Despite this, current optimization frameworks such as KORDI for cloud computing are intended to solve these problems but do not significantly overcome the difficulties associated with real-time performance tuning [39].
- **Complexity of Optimization:** Many existing optimization methods require deep integration with the underlying computation and communication libraries, making them complex and prone to implementation errors. The complexity increases the likelihood of mistakes during deployment, which can negatively impact performance [40].
- **Scalability Challenges:** While Spark is designed for scalability, maintaining linear scalability as datasets grow in size and models increase in complexity is still a challenge. There have been techniques, such as hierarchical parameter synchronization, utilized to solve the above problems, but due to the complexity of the techniques and the fact that they are not versatile enough to cater to all types of Spark applications, the above issues pose constraints. [15,41,42].
- **Need for Performance Tuning:** Modern Apache Spark users do not only have problems with fine-tuning the application's performance but also witness the corresponding evolution of Apache Spark. Some APIs become deprecated, and there is no clear set of rules for performance optimization most of the time, which leads to poor application performance. According to the users, the efficiency tuning of Spark MLlib jobs is often challenging if one does not possess tremendous background information on the underlying framework [43].

To conclude, we have encountered several optimization opportunities in distributed ML frameworks, as well as particular improvements in MLlib algorithms; however, there are still some diverse difficulties. The diverse types of optimizations, size and efficiency problems, and recurrent demands of fine-tuning all point to the need for more studies in this area.

2.5. Gaps in the Literature on Predictive Modeling for Performance Optimization in Apache Spark MLlib

While investigating the current state of the art of using MLlib for performance prediction in the context of Apache Spark MLlib concerning performance improvement, several research gaps have been noted. These gaps are important for developing more efficient and easily applicable large-scale models in big data models.

Unlike the previous studies that have extended only in regards to static optimization or limited scalability [16], this study extends the literature by providing a dynamic and predictive modeling framework that is responsive to different workload requirements.

For example, Tsai et al. concentrate on the memory optimization of fixed service level agreements [14], while our approach uses machine learning techniques to dynamically forecast the execution times and resource requirements. Similarly, as opposed to the study by Gulino et al. where the research was aimed at predicting the performance of workflows applications [13], instead, our approach is developed to specifically optimize and enhance the performance of the MLlib operators to be deployed within the distributed environment with a focus on the crosscutting concerns of structured and unstructured data workloads.

2.5.1. Current Gaps

Hyperparameter Optimization Techniques

One of the biggest drawbacks of the existing work in the field is the use of black box techniques for hyperparameter tuning. Such approaches become inefficient because they often do not consider the mechanism by which data are computed within Spark MLlib, thus obtaining flawed information. For this purpose, there is a strong requirement to find out techniques other than model-free reinforcement learning or adaptive optimization strategies that can handle the reformation in Distributed Systems regarding hyperparameter tuning [19].

Understanding Computational Resource Utilization

While it is understood that CPU-bound issues and memory limitations influence Spark MLlib models' performance, existing studies do not offer a detailed analysis of how different computational resources—CPU, memory, and I/O—impact the overall model performance. High efforts are still necessary in studying the allocation of resources in distributed systems to make the best of the available resources for use in Spark applications to achieve the best performance [44].

Integration of External Factors

Most of the prior work is restricted to discussing the impact of external factors including institutional pressures, organizational culture, and structural constraints regarding the use of predictive modeling in big data environments. Integrating such externality into prediction environments may offer a needed and more realistic view of these externalities and help in enhancing the application of the models in various settings [45].

Scalability and Real-World Application

Several research studies in this field are still limited to theoretical frameworks or pilot studies. This gives rise to a research question on the real-world scalability of a predictive model and especially on how well these techniques work in real-world big data scenarios. This is due, first, to the fact that the performance of models on the given data as a function of data size, type of computing environment, and application loads of actual large-scale real-world applications have yet to be investigated [46,47].

Thus, it is important to fill these gaps in the literature for the contribution to predictive model development and improvement of the performance of the Apache Spark MLlib. As such, it is recommended that ongoing and future research seeks to engage in more scalable and comprehensive assessments, consider fresh means of optimization in resource usage [48], and factor in more external variables that will make these models more applicable in the real world of Big Data Analytics.

Future and ongoing research should focus on developing user-friendly tools, for example, graphical interfaces for Spark MLlib, so it is easier to apply machine learning algorithms. It could encourage wider take up among real users, who are more varied in their levels of experience. SparkReact is a tool for fabricating web applications on React on top of Apache Spark [49]. It combines Spark's real-time data processing abilities with its

impactful interactive user interfaces. This integration facilitates application development for big datasets with an easy-to-use interface.

2.6. Feature Engineering Techniques in Apache Spark MLlib

Emerging research shows that feature engineering approaches widely used in Apache Spark MLlib enhance the efficacy of various fields. For example, a study on heart disease prediction proposed the Principal Component Heart Failure (PCHF) approach which improved the detection rate by concentrating on the most important characteristics [50]. In the domain of cybersecurity, for instance, MLlib provided within Apache Spark was used for feature extraction using K-Means Clustering for IDS in detecting the abnormality of data and demonstrating the capability of the algorithm in the security domain [51]. Furthermore, diabetes prediction used Spark MLlib to evaluate the performance of different machine learning algorithms and demonstrated that the Random Forest algorithm yields high accuracy, underlining the penetration of the library in analytics of health data.

Further, the feature engineering capacity of Spark MLlib has been used to process large-scale text data with a considerable increase in run time compared to SparkSQL using the RDD API [52]. Such improvements also highlight Spark MLlib's Effective Feature Engineering tools and show how big data with applications in healthcare, cybersecurity, and all sectors can be analyzed.

2.7. Rationale for Model Selection

The Random Forest and the Gradient Boosting techniques were selected over other methods because they perform well on varieties of predictive tasks, and are robust methods in handling nonlinearity in the data with large samples. Gradient Boosting tends to outperform Random Forest in determining through ensemble learning the more accurate predictions, while Random Forests tend to be effective in minimizing overfitting. The resource utilization pattern observed in Apache Spark MLlib operators was complex enough to make these models preferred over simpler alternatives like Linear Regression. However, we included simpler models such as Linear Regression for baselines as a comparison to highlight the power of these advanced techniques.

3. Methodology

The proposed methodology for predicting the performance of machine learning (ML) operators in Apache Spark is illustrated in Figure 1. The process launches with the foundation of the infrastructure (Step 1), during which a multi-node Apache Spark cluster is developed. This cluster brings together a leader node and a range of follower nodes, all linked via a private network, which provides the ability for distributed data processing. For the data creation in Step 2, we use data generators specifically for some of the ML algorithms such as K-Means, Random Forest, and Word2Vec. The datasets, in terms of their size and dimensions, allow for a wide range of inputs to assess operator performance.

After creating the dataset, in Step 3, the ML operators run with different parameters and collect metrics inclusive of execution time, CPU utilization, and memory usage. The collection of these performance metrics that show operator resource use and time efficiency is carried out while the execution is in progress; they serve as the groundwork for model training. In Step 4, the metrics go into a prediction model in Step 5, where they are trained to project the performance of ML operators according to the dimensions of the input dataset. The final model predicts important performance indicators, including execution time and memory usage, giving the ability to optimize and plan computational resources for future operations.

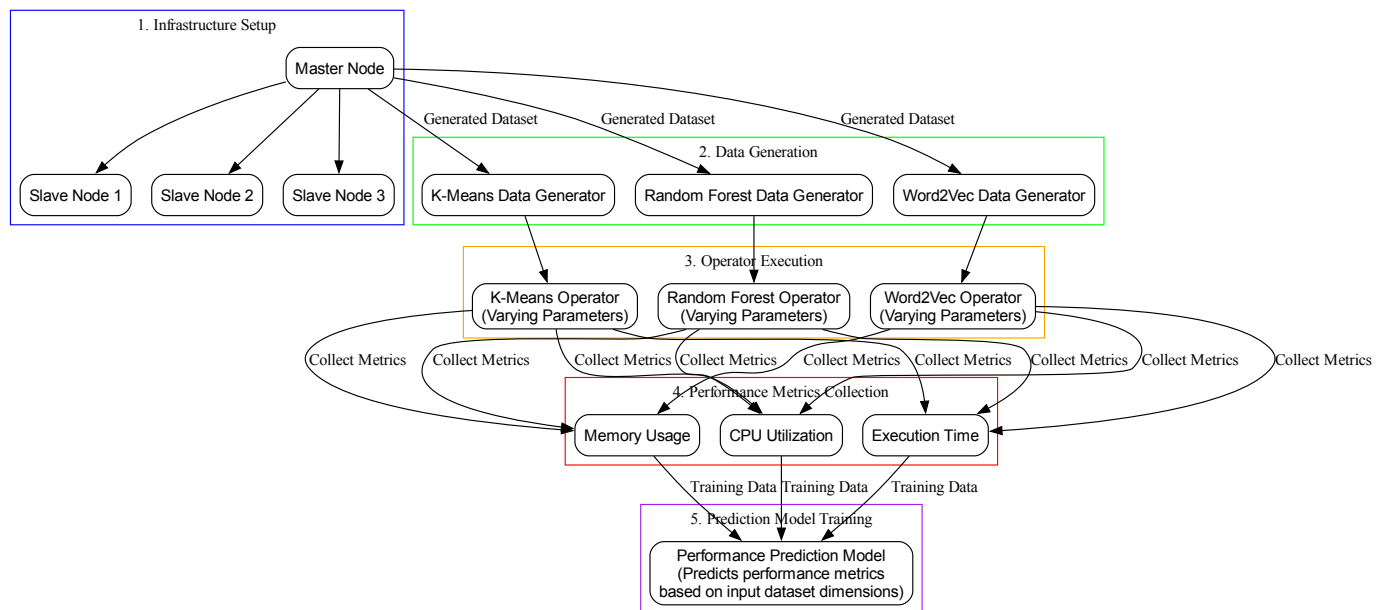


Figure 1. Illustration of the methodology.

Figure 1 offers an overarching view of the system architecture, while Figure 2 shows the internal architecture of the workflow for dataset preparation, training, and validation. The latter diagram emphasizes the logical steps, starting from synthetic dataset generation and culminating in model training and performance evaluation, effectively complementing the high-level depiction in the former.

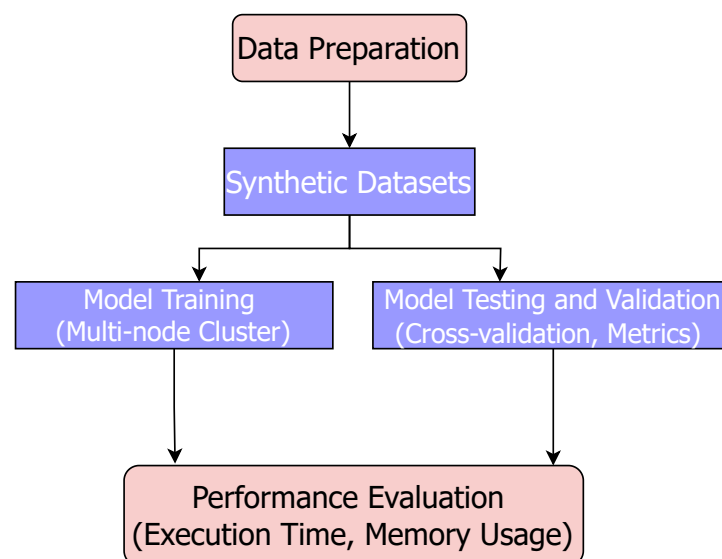


Figure 2. Workflow for dataset preparation, training, and validation.

In Step 3 of Figure 1, the ML operators are executed with different parameters and the metrics include execution time, CPU, and memory usage. However, the collection of these performance metrics indicating operator resource use and time efficiency are performed in progress of the execution and form the basis for model training. In Step 5, the metrics obtain predicted in a model to learn how to predict the performance of ML operators on the basis of the dimensions of the input dataset in Step 4. It also predicts the execution time and memory usage of important performance indicators in order to allow the planner to optimize and plan computational schedules for future operations.

3.1. Infrastructure Setup

This section describes the configuration of a multi-node Apache Spark cluster hosted on the DIOGENIS service at the Computer Engineering and Informatics Department at the University of Patras, in Greece. The cluster consists of one leader node and one follower node, represented as $N = \{M_1, S_1\}$, where M_1 is the leader node and S_1 is the follower node. The primary objective of this setup is to distribute computational tasks efficiently for large-scale data processing using Spark's in-built distributed computation model.

Given a machine learning operator O , the goal is to calculate its performance in a specific scenario P . We aim to evaluate the performance by measuring two metrics: total processing time and memory usage. These are modeled as functions of the underlying infrastructure and data distribution across the nodes, and will be evaluated later using the performance space $\mathbb{R}^2$, where each point $p \in \mathbb{R}^2$ is a tuple of (execution time, memory usage).

3.2. Cluster Architecture and Mathematical Model

Apache Spark adopts a leader–follower architecture for distributed computing, which can be formally represented as a graph $G = (V, E)$, where

- $V = \{M_1, S_1\}$ are the set of nodes, with M_1 as the leader node and S_1 as the follower node.
- $E = \{e_{M_1 \rightarrow S_1}\}$ represents the directed edges, where $e_{M_1 \rightarrow S_1}$ symbolizes task distribution from the leader to the follower node.

The leader node M_1 coordinates task execution by assigning computational jobs J , denoted as $J = \{J_1, J_2, \dots, J_n\}$, to follower nodes, distributing them across the available resources. Formally, the resource allocation to node S_1 can be represented as follows:

$$\text{Resources}(S_1) = \{\text{CPU}(S_1), \text{RAM}(S_1)\}$$

Each job J_i has an associated computation time t_i and memory requirement m_i , leading to the evaluation of performance metrics. Let T_{total} represent the total computation time, and M_{total} represent the total memory usage across all jobs J executed by the cluster.

3.2.1. Leader–Follower Task Assignment

The leader node M_1 dispatches tasks to the follower nodes based on available resources, which can be mathematically modeled as a function f_{assign} , where

$$f_{\text{assign}}(M_1, S_1) = \begin{cases} J_1 & \text{if CPU and RAM are available on } S_1, \\ 0 & \text{otherwise.} \end{cases}$$

The processing capability of S_1 can be denoted as $C(S_1)$, which is the total computational power available, and is a function of CPU cores and memory:

$$C(S_1) = \text{CPU}(S_1) \times \text{RAM}(S_1)$$

Thus, the execution time for each job J_i can be expressed as follows:

$$t_i = \frac{J_i}{C(S_1)}$$

The overall execution time for the cluster, T_{total} , for a given scenario P is as follows:

$$T_{\text{total}} = \sum_{i=1}^n t_i = \sum_{i=1}^n \frac{J_i}{C(S_1)}$$

where n is the number of jobs distributed across the nodes.

3.2.2. Performance Metrics: Time and Memory Usage in $\mathbb{R}^2$

To evaluate the performance of the operator O in scenario P , we calculate both the total time T_{total} and memory usage M_{total} . These are represented in the performance space $\mathbb{R}^2$, where

$$p_{\text{performance}} = (T_{\text{total}}, M_{\text{total}}) \in \mathbb{R}^2$$

The memory usage for the entire cluster is given by

$$M_{\text{total}} = \sum_{i=1}^n m_i$$

where m_i is the memory footprint of each job J_i . Therefore, the performance of operator O for scenario P can be summarized as follows:

$$p_{\text{performance}} = \left(\sum_{i=1}^n \frac{J_i}{C(S_1)}, \sum_{i=1}^n m_i \right)$$

3.3. Data Partitioning and Task Distribution

In Apache Spark, data are divided into partitions, each representing a subset of the dataset distributed across worker nodes. Let D be the dataset, and $P(D)$ be the set of partitions:

$$P(D) = \{D_1, D_2, \dots, D_k\}$$

where k is the total number of partitions. Each partition D_i is processed independently by the worker nodes, enabling parallel execution. The relation between partitions and workers can be modeled as follows:

$$f_{\text{partition}}(P(D), W) = \frac{k}{W}$$

where W represents the number of available workers in the cluster, and $\frac{k}{W}$ is the partition-to-worker ratio. For our two-node setup ($W = 1$ worker on S_1), the partition processing time is directly proportional to the total execution time:

$$T_{\text{partition}} = \sum_{i=1}^k \frac{D_i}{C(S_1)}$$

Thus, the total time T_{total} is the sum of partition processing times, which also accounts for Spark's task parallelism.

3.4. Goal of Evaluation

The ultimate objective of this study is to evaluate the performance of the operator O on scenario P by analyzing $p_{\text{performance}}$ in $\mathbb{R}^2$. The evaluation criteria include total execution time and memory consumption as defined above. Later sections will provide empirical results and further evaluate how these metrics behave under different configurations of Spark and varying data sizes.

3.5. Initialization

In this phase, we initialized the Spark session and made necessary modifications to its default configuration options, based on a comprehensive analysis of system performance and technical constraints. We aimed to modify these instructions to optimize resource usage and to guarantee the optimal execution of O in the experiment P .

Let the total memory allocated to each executor Mem_{exec} be represented as follows:

$$\text{spark.executor.memory} = 2000 \text{ MB}$$

For example, Mem_{exec} refers to the maximum amount of memory that each Spark executor can access. The setting we run with makes sure that the executors are all working within the resource boundaries of existing virtual machines (VMs), thus getting the best out of them before burdening the system. Additionally, we assigned each executor a number of cores $\text{Cores}_{\text{exec}}$:

$$\text{Cores}_{\text{exec}} = \text{spark.executor.cores} = 2$$

The driver memory $\text{Mem}_{\text{driver}}$ was set to the following:

$$\text{Mem}_{\text{driver}} = \text{spark.driver.memory} = 3 \text{ GB}$$

This value was determined through empirical testing, as smaller values significantly degraded system performance. Furthermore, we reduced the periodic garbage collection interval $\text{GC}_{\text{interval}}$ to

$$\text{GC}_{\text{interval}} = \text{spark.cleaner.periodicGC.interval} = 1 \text{ minute}$$

This adjustment helped to mitigate the risk of memory overload by forcing frequent garbage collection, ensuring unused memory was quickly reclaimed.

3.6. Dataset Generation

The dataset generation process was a critical component of our experimental setup, allowing us to evaluate operator performance O across various dataset configurations. Let the dataset D be characterized by the number of samples n and the number of features f . We generated datasets dynamically based on these parameters, with the goal of controlling the scale and dimensionality of the data.

For the KMeans and RandomForest operators, we used standard functions from the `sklearn.datasets` package, specifically the following:

$$D_{\text{KMeans}} = \text{make_blobs}(n, f, \text{clusters})$$

$$D_{\text{RandomForest}} = \text{make_regression}(n, f)$$

where D_{KMeans} and $D_{\text{RandomForest}}$ represent the datasets generated for the KMeans and RandomForest algorithms, respectively. The number of clusters c in KMeans was a configurable parameter, ranging from 2 to 20.

For the Word2Vec operator, we generated datasets of sentences, each sentence being a random sequence of words. The dataset size D_{Word2Vec} is represented as follows:

$$D_{\text{Word2Vec}} = \{\text{Sentence}_1, \text{Sentence}_2, \dots, \text{Sentence}_n\}$$

Each sentence contains a fixed number of words w , where $w = 15\text{--}50$, based on linguistic research on average sentence length. The sentences were generated by randomly sampling from a dictionary of 200 unique words. The generated datasets were saved in CSV format, and their metadata, including n and f , were stored in separate files.

3.7. Dataset Sources and Relevance

This study exploited synthetic datasets designed to enable the controlled experimental performance of Apache Spark operators, which were used in this work. These datasets

were of particular importance, which allowed for strict control of parameters—size, feature dimensionality, and complexity—and permitted a comprehensive study of computational performance and scalability.

- **Synthetic Datasets:** Generated using Python’s 3.10 version, scikit-learn library for key MLlib operators:
 - `make_blobs` for K-Means Clustering: Configured to test scalability across varying numbers of clusters and data dimensions.
 - `make_regression` for Random Forest: Designed to simulate regression problems with tunable complexity.
 - Custom sentence generators for Word2Vec: Created sequences of words with varying sentence lengths, mimicking natural language text.

The systematic exploration of operator behavior over multiple configurations was possible using synthetic datasets. While we did not use real-world datasets directly, the synthetic datasets were generated to resemble real-world behavior in the machine learning workflows, including clustering, regression, and text processing. The approach offers a solid basis for understanding operator performance in controlled situations, extending to actual use in real-world situations in the future.

3.8. Model Training

Apache Spark MLlib’s operators, including K-Means, Random Forest, and Word2Vec, were configured with varied hyperparameters:

- **K-Means:** Number of clusters (c) varied from 2 to 20.
- **Random Forest:** Tree depths ranged from 5 to 50.
- **Word2Vec:** Sentence lengths varied between 15 and 50 words.

Training was conducted on a distributed multi-node Spark cluster to ensure scalability.

3.9. Model Testing and Validation

The key metrics for evaluation were as follows:

- **Execution Time:** Measured as the elapsed time to complete training.
- **Memory Usage:** Monitored through Spark’s REST API to assess resource consumption.

Cross-validation was employed to ensure reliability of predictions.

3.10. Execution

The execution phase involved running each operator O on various datasets D , while adjusting the number of rows n and columns f . The operator execution time $T(O)$ and memory usage $M(O)$ were the primary metrics of interest. For each operator, the number of rows n and columns f were varied according to the ranges of the datasets. The performance of KMeans depends on both n , f , and the number of centers c :

$$T(O_{\text{KMeans}}) \propto n \times f \times c$$

For RandomForest, execution time $T(O_{\text{RandomForest}})$ scales with the number of trees t and the dataset dimensions:

$$T(O_{\text{RandomForest}}) \propto n \times f \times t$$

In the case of Word2Vec, the execution time $T(O_{\text{Word2Vec}})$ depends on the length of the sentences w and the number of rows n :

$$T(O_{\text{Word2Vec}}) \propto n \times w$$

All models were fit to the datasets without performing predictions, as the goal was to evaluate the training time and memory consumption of each operator. The number of iterations for KMeans was fixed at 10 to avoid convergence failures. Throughout the execution phase, system resources, including CPU usage and memory consumption, were closely monitored, and garbage collection was manually triggered to free up memory when necessary.

3.11. Statistics Collection

The final phase of each experiment involved logging and preserving key statistics to be used in the performance evaluation of the operators. The most significant metrics collected were execution time $T(O)$ and memory usage $M(O)$.

Let the dataset metadata $\text{Meta}(D)$ be represented as follows:

$$\text{Meta}(D) = (n, f, \text{Size}_{\text{MB}}, \text{Clusters})$$

where Size_{MB} is the size of the dataset in megabytes, and Clusters is the number of clusters for KMeans datasets.

The training time $T(O)$ was measured using Python's time module and included only the time taken for the operator to fit the dataset:

$$T(O) = \text{End_time} - \text{Start_time}$$

Memory usage $M(O)$ was collected via the Spark UI REST API by aggregating the memory usage across all executors:

$$M(O) = \sum_{i=1}^e \text{memoryUsed}_i$$

where e is the number of executors. The collected metrics were stored and later used for performance analysis.

In conclusion, the two key metrics—execution time and memory usage—were the most representative values for assessing the performance of each operator. These values formed the foundation for predicting operator behavior and optimizing the Spark cluster's configuration.

3.12. Rationale for Metrics

Execution time and memory usage were chosen as the main metrics due to their direct impact on the scalability and efficiency of distributed systems. These metrics were chosen for the following reasons:

- They serve as industry-standard indicators for evaluating computational performance.
- They help to identify bottlenecks in resource allocation and operator configuration.

Thus, the metrics proved essential in building predictive models for Apache Spark configuration optimization as well as overall system performance.

4. Experimental Results

In this section, we provided the results obtained from the experimental evaluation of these operators namely KMeans, RandomForest, and Word2Vec, amongst others. In each experiment, the operator O was tested concerning the input dataset D and characteristics of the number of samples n , the number of features f , and the size of the dataset S_D . The acquired datasets that have more than one feature and target column were then utilized for analysis and model prediction.

1. **Total Time:** The time $T(O)$ that the operator took to fit the data.
2. **Memory Usage:** The amount of memory $M(O)$ used by the cluster.
3. **num_samples:** The number of samples n in the dataset.
4. **num_features:** The number of features f in each sample.
5. **num_classes:** The number of clusters c (only applicable for KMeans).
6. **dataset_size:** The dataset size S_D in megabytes.

The first two columns (Total Time and Memory Usage) serve as target variables for our prediction models, while the others are input features. Further to identifying the given input features, we examined the correlation between these input features and the target variables to better understand the experimental findings through data visualization techniques.

4.1. KMeans Operator

The KMeans operator O_{KMeans} was used with a set of different datasets to investigate how the number of samples n , features f , and clusters c influenced the overall computation time $T(O)$ and memory requirements $M(O)$.

4.1.1. Total Time

The relationship between total time $T(O_{\text{KMeans}})$ and the number of samples n is illustrated in Figure 3. As expected, the time complexity of the KMeans algorithm grows linearly with the number of samples, as

$$T(O_{\text{KMeans}}) \propto n \times f$$

This is consistent with the time complexity of KMeans, which is $O(n \cdot k \cdot f)$, where k represents the number of clusters, n the number of samples, and f the number of features. Figure 3 shows the linear trend for both samples and features.

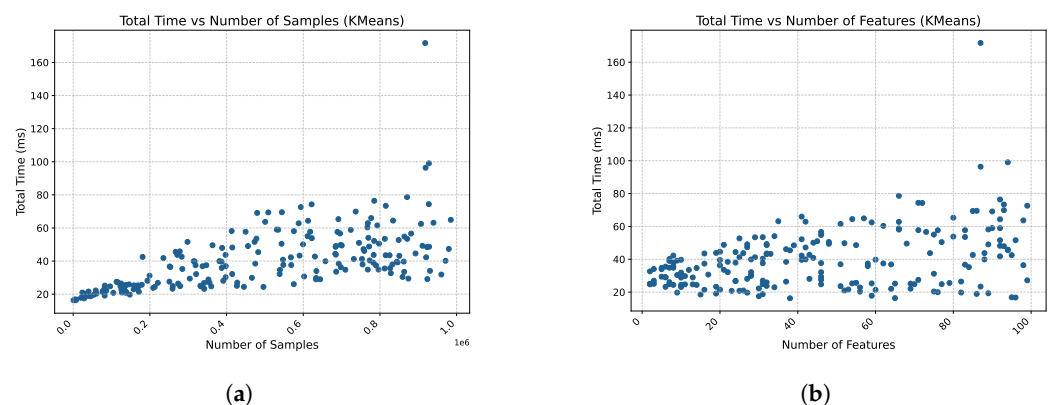


Figure 3. Total time for KMeans in relation to number of samples and features. (a) Relationship between total time and number of samples for K-Means. (b) Total time and number of features relationship for KMeans.

The linear increase in time with sample size demonstrates the scalability of the algorithm under varying dataset sizes in Figure 3a.

Figure 4 shows that the number of clusters c has minimal impact on total time. On the other hand, the dataset size S_D has a strong linear correlation with time, suggesting that the larger the dataset, the more significant the processing time.

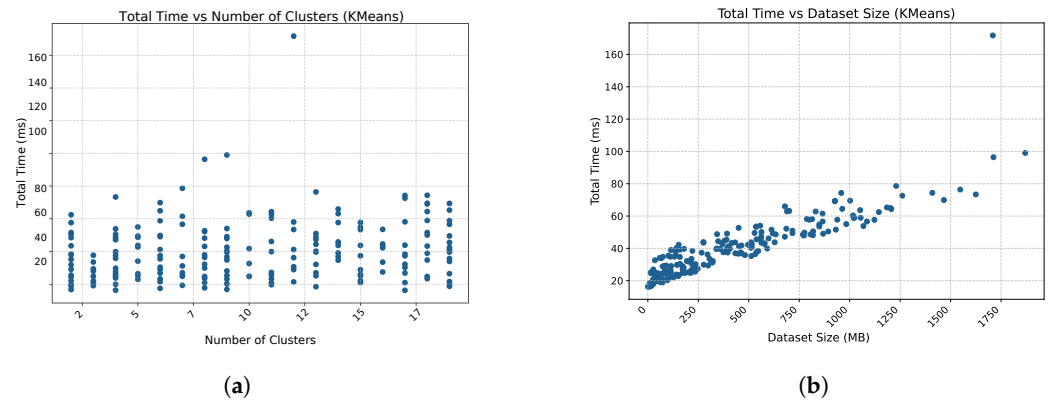


Figure 4. Total time for KMeans in relation to number of clusters and dataset size. (a) Total time and number of clusters relationship for KMeans. (b) Total time and dataset size relationship for KMeans.

4.1.2. Memory Usage

Similarly, memory usage $M(O_{KMeans})$ was analyzed with respect to input features. Figure 5 shows the memory usage scales with the number of samples n and features f .

$$M(O_{KMeans}) \propto n \quad \text{and} \quad M(O_{KMeans}) \propto f$$

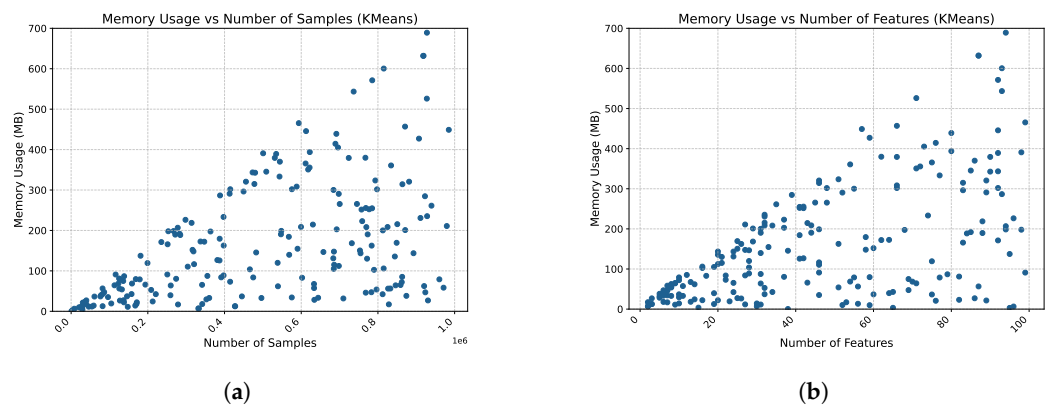


Figure 5. Memory usage for KMeans in relation to number of samples and features. (a) Memory usage and number of samples relationship for KMeans. (b) Memory usage and number of features relationship for KMeans.

Figure 6 shows that while the number of clusters c has little impact on memory usage, the dataset size S_D shows a near-linear relationship with memory usage.

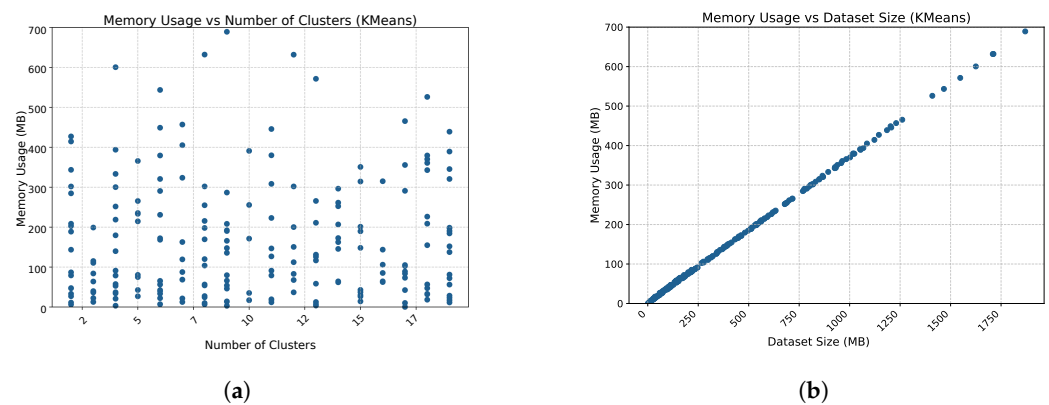


Figure 6. Memory usage for KMeans in relation to number of clusters and dataset size. (a) Memory usage and number of clusters relationship for KMeans. (b) Memory usage and dataset size relationship for KMeans.

4.2. Random Forest Operator

The RandomForest operator O_{RF} was evaluated using the same methodology. The total time $T(O_{RF})$ was found to exhibit linear growth with both the number of samples and features, similar to KMeans:

$$T(O_{RF}) \propto n \times f$$

Figure 7 confirms these relationships. Additionally, as shown in Figure 8, the time required increases as the number of samples is getting bigger, while the memory usage follows a similar pattern. As per the memory usage of the Random Forest Method, the results are shown in Figure 9.

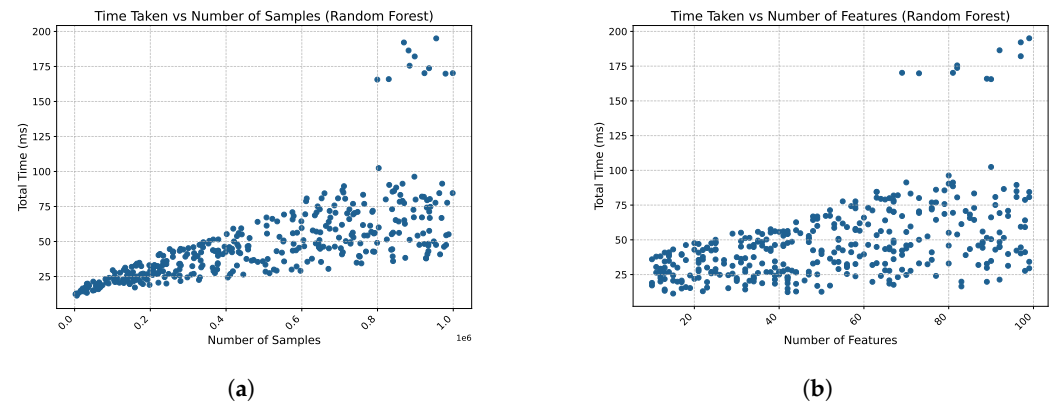


Figure 7. Time taken for Random Forest in relation to number of samples and features. (a) Time taken and number of samples relationship for Random Forest. (b) Time taken and number of features relationship for Random Forest.

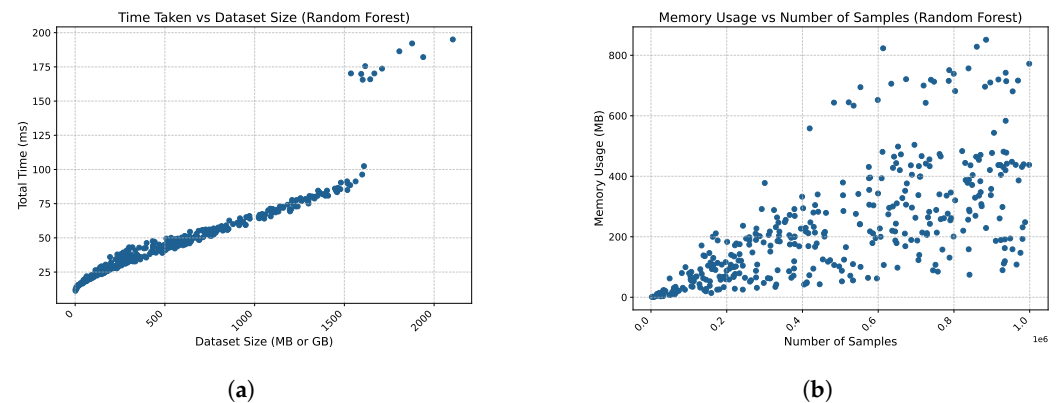


Figure 8. Time taken and memory usage for Random Forest in relation to dataset size and number of samples. (a) Time taken and dataset size relationship for Random Forest. (b) Memory usage and number of samples relationship for Random Forest.

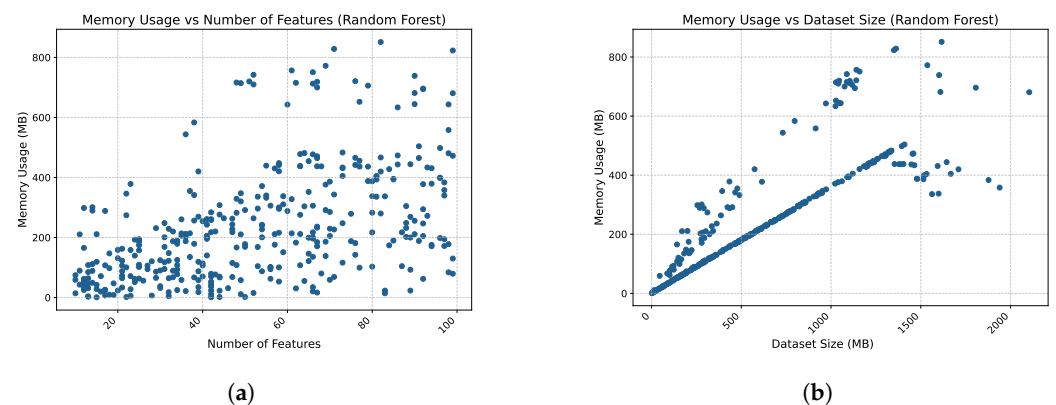


Figure 9. Memory usage for Random Forest in relation to number of features and dataset size. (a) Memory usage and number of features relationship for Random Forest. (b) Memory usage and dataset size relationship for Random Forest.

4.3. Word2Vec Dataset

The results of the performance analysis for the Word2Vec model are presented in Figure 10. The first set of plots illustrates the relationship between total time and the number of samples and features, as shown in Figure 10a,b. These plots demonstrate a clear upward trend in time consumption as both the number of samples and features increase, highlighting the computational demands of the Word2Vec method. Additionally, Figures 11 and 12 depict the memory usage in relation to the number of samples, features, and dataset size. These figures reveal the substantial memory requirements of Word2Vec, further emphasizing its complexity. The plots collectively provide a comprehensive understanding of how the Word2Vec operator scales with increasing data size and feature dimensions.

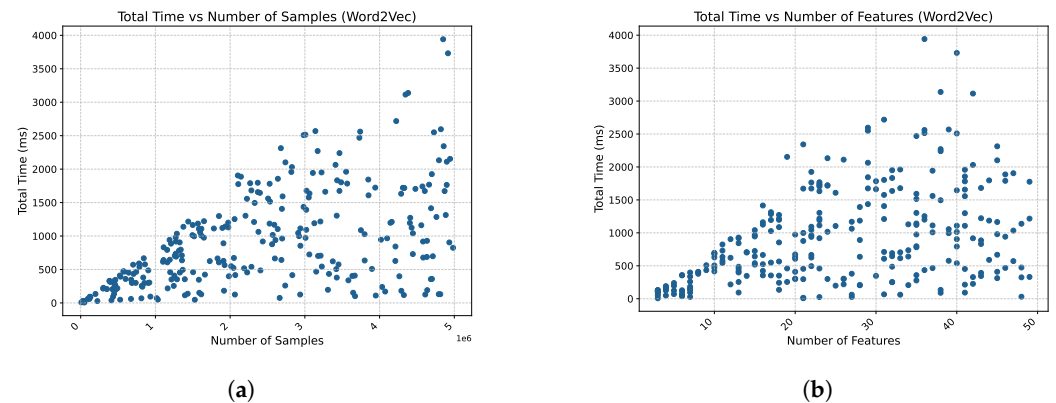


Figure 10. Total time performance metrics for Word2Vec. (a) Total time vs. number of samples (Word2Vec). (b) Total time vs. number of features (Word2Vec).

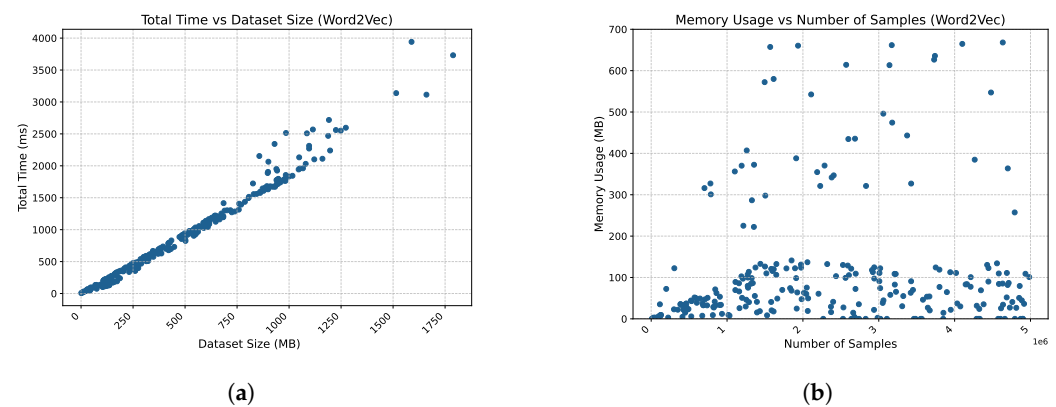


Figure 11. Memory usage performance metrics for Word2Vec (Part 1). (a) Total time vs. dataset size. (b) Memory usage vs. number of samples.

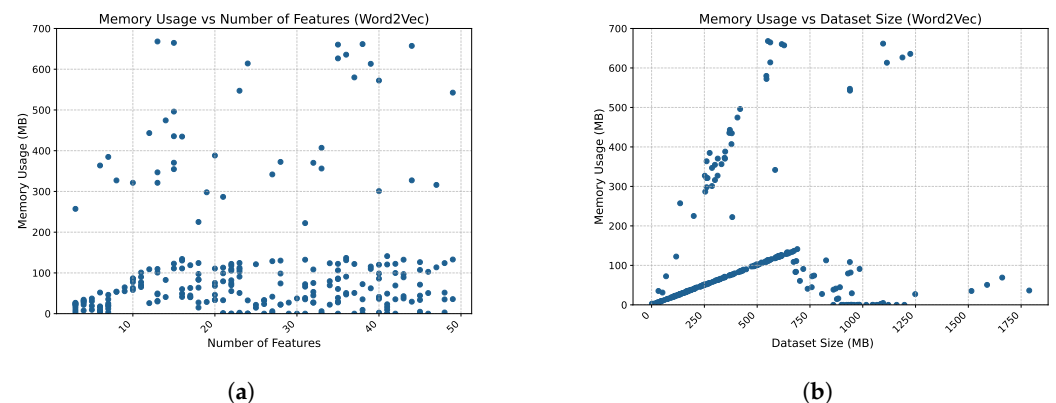


Figure 12. Memory usage performance metrics for Word2Vec (Part 2). (a) Memory usage vs. number of features. (b) Memory usage vs. dataset size.

4.4. Model Performance Comparison

We compared the performance of different models by evaluating metrics such as mean absolute error (MAE) and R^2 . The R^2 metric measures the proportion of the variance in the dependent variable that is predictable from the independent variable, while the mean absolute error (MAE) measures the average absolute difference between the predicted and actual values.

In order to provide a more comprehensive evaluation of model accuracy, we also included Mean Squared Error (MSE) as an extra performance metric alongside R^2 and MAE. A squared perspective on prediction errors is offered by MSE which penalizes larger deviations more. The byproduct of this metric is to give us a sense of how the errors are distributed and which values are outliers. The results indicate that Gradient Boosting performed best over all the experiments, confirming its suitability for modeling resource utilization patterns in Apache Spark MLlib.

4.4.1. Random Forest Regression

We plotted the R^2 scores and MAE results in separate graphs. As seen in the graph for total time, the Random Forest Regressor achieved the highest R^2 score, closely followed by the Gradient Boosting Regressor. The other models lagged significantly behind in terms of performance. The results for the total time of the Random Forest are shown in Figure 13, while the memory usage in Figure 14.

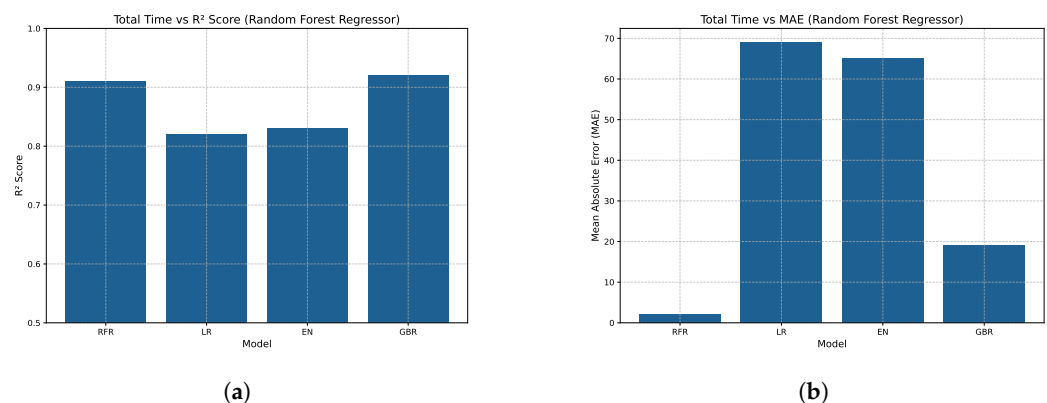


Figure 13. Total time performance metrics for Random Forest Regressor. (a) Total time vs. R^2 score for Random Forest Regressor. (b) Total time vs. MAE for Random Forest Regressor.

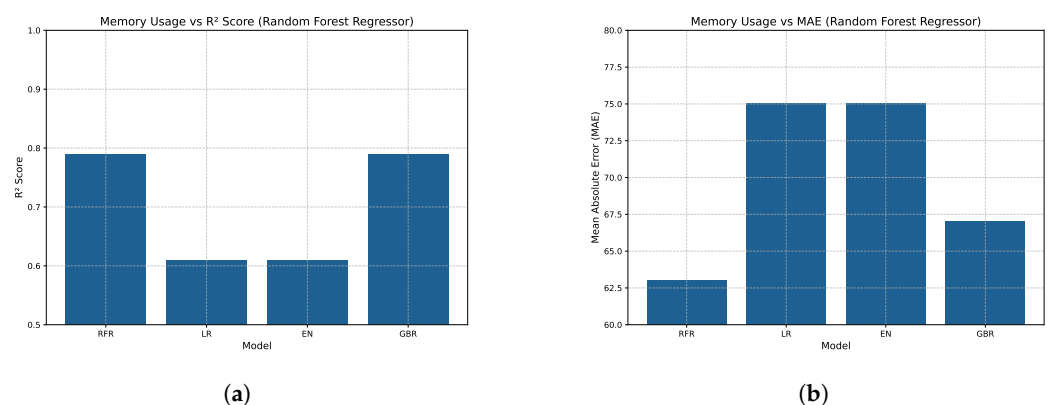


Figure 14. Memory usage performance metrics for Random Forest Regressor. (a) Memory usage vs. R^2 score for Random Forest Regressor. (b) Memory usage vs. MAE for Random Forest Regressor.

Similarly, in the graph for memory usage, the Random Forest Regressor and the Gradient Boosting Regressor stood out as the top-performing models, with the former outperforming the latter. Once again, the other models were far behind in terms of performance.

After running numerous tests, we found that the Random Forest Regressor had much more stable results across multiple runs. Based on its superior performance, we decided to use the Random Forest Regressor as our final model.

4.4.2. KMeans Regression

The results from the evaluation of total time performance suggest that the Gradient Boosting Regressor exhibits better performance for both R^2 and MAE metrics compared to the Random Forest Regressor. Among the evaluated models, these two demonstrate the highest accuracy in terms of performance. Conversely, the remaining models exhibit lower levels of accuracy. More specifically, the results for the total time of KMeans method are given in Figure 15, while the results for the memory usage in Figure 16.

For memory usage, the evaluated models showed similar performance, with no significant differences. Based on the evaluation results for both total time and memory usage, the Gradient Boosting Regressor exhibits the best overall performance.

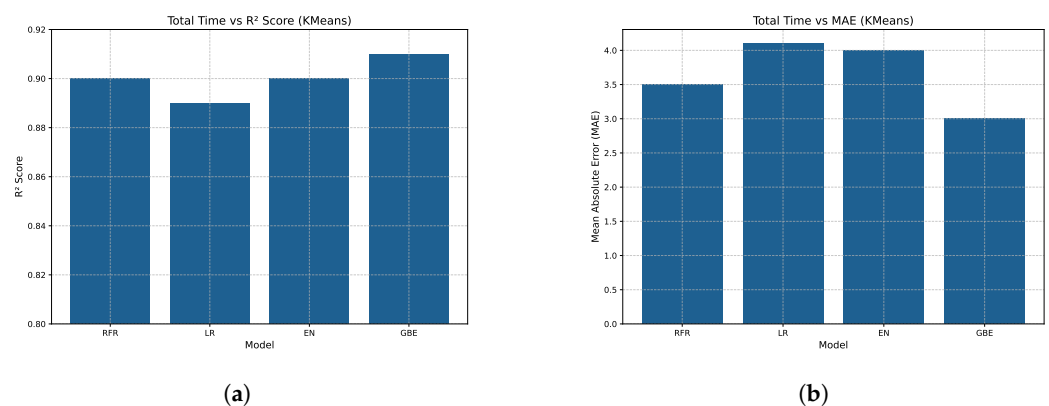


Figure 15. Total time performance metrics for KMeans. (a) Total time vs. R^2 score for KMeans. (b) Total time vs. MAE for KMeans.

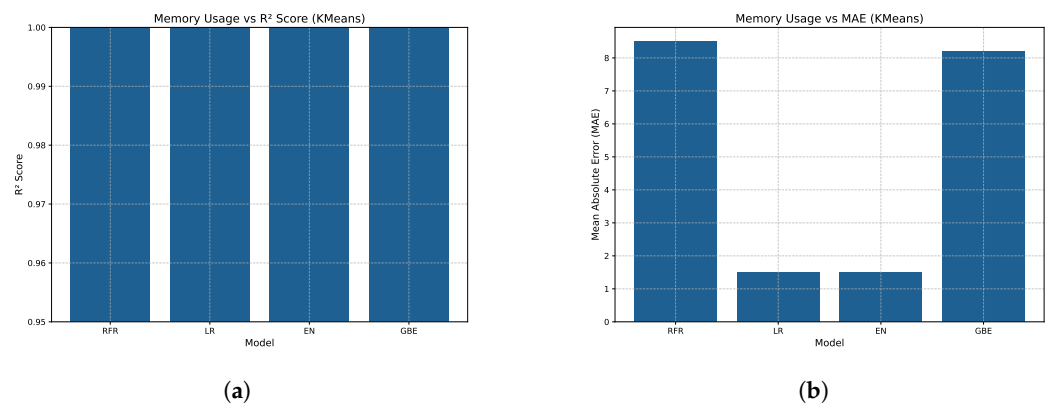


Figure 16. Memory usage performance metrics for KMeans. (a) Memory usage vs. R^2 score for KMeans. (b) Memory usage vs. MAE for KMeans.

4.4.3. Word2Vec Regression

The evaluation of the total time performance revealed that the Linear Regression model outperformed the other models in terms of both R^2 and MAE metrics. However, as previously mentioned, the data points for memory usage were insufficient to train an accurate model. Thus, the performance for both R^2 and MAE is notably low, as shown in the Figures 17 and 18.

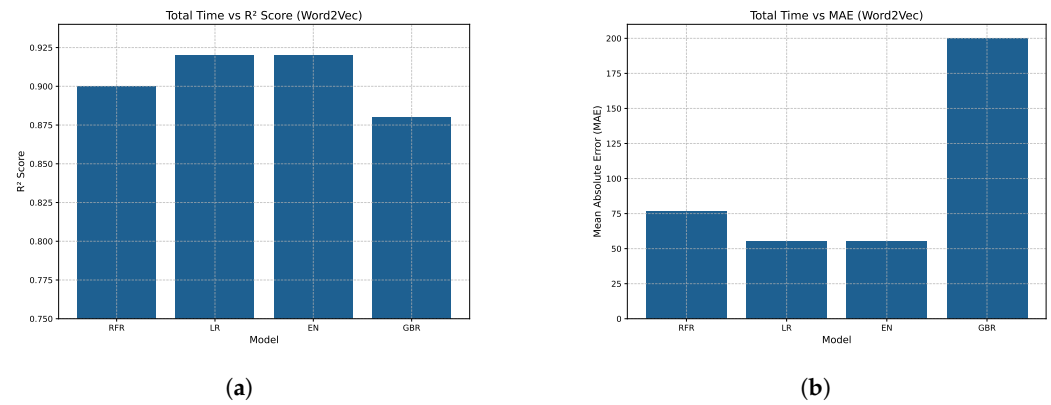


Figure 17. Total time performance metrics for Word2Vec. (a) Total time vs. R^2 score for Word2Vec. (b) Total time vs. MAE for Word2Vec.

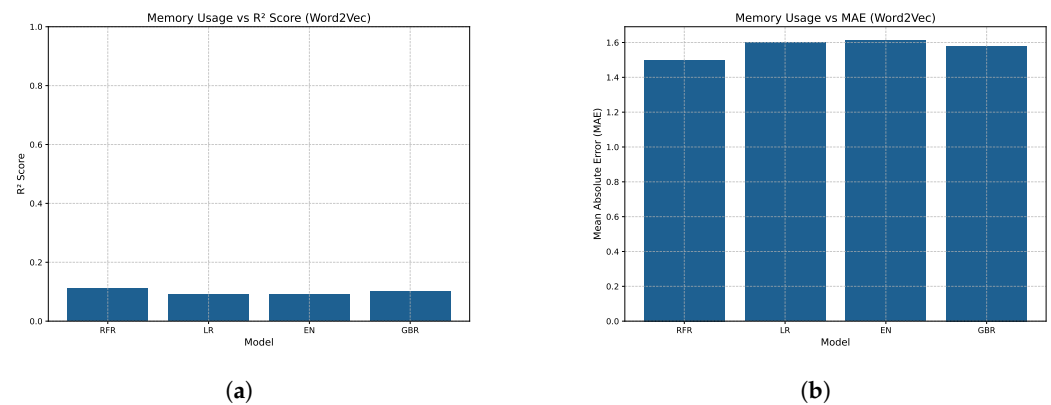


Figure 18. Memory usage performance metrics for Word2Vec. (a) Memory usage vs. R^2 score for Word2Vec. (b) Memory usage vs. MAE for Word2Vec.

4.5. Limitations of Word2Vec and Recommendations

The memory consumption predictions for the Word2Vec model displayed inconsistencies, as evidenced by a lower R^2 value (0.06). This discrepancy can be attributed to the complexity of the algorithm and the irregular memory allocation patterns caused by processing unstructured textual data.

To address this, we recommend integrating advanced memory profiling tools and developing enhanced feature engineering techniques such as the following:

- Incorporating word frequency distributions and text complexity metrics as additional input features for predictive modeling.
- Adopting hierarchical or hybrid modeling approaches to capture the nonlinear behavior inherent in text-based algorithms.

These strategies aim to improve the accuracy of memory usage predictions for Word2Vec, thereby enhancing its applicability in large-scale distributed systems.

4.6. Statistical Validation of Experimental Results

To evaluate the statistical significance of the observed differences in execution time and memory usage across the models (K-Means, Random Forest, and Word2Vec), we calculated the confidence intervals (CI) at the 95% level and conducted hypothesis tests using ANOVA for multi-group comparisons. The resulting p -values were below 0.05, indicating significant differences in the performance metrics across models.

For instance, the Random Forest model demonstrated a significant reduction in memory usage compared to K-Means and Word2Vec, with a 95% CI ranging from 12.4% to 18.9%. These statistical validations provide greater confidence in the reported improvements and demonstrate the robustness of the proposed predictive modeling framework.

5. Discussion

In this study, we explored the inner workings of Apache Spark’s MLlib library, which provides a range of algorithms and tools for processing large datasets and building machine learning models. Our focus was on three key operators: K-Means Clustering, Random Forest Regressor, and Word2Vec. We trained several models to predict the performance of each operator based on two key metrics: total execution time and memory usage. The results of the best-performing models for each operator are summarized in Table 2.

Table 2. Best models and corresponding metrics for each operator.

Operator	Best Model	Total Time R^2	Memory Usage R^2
K-Means	Gradient Boosting Regressor	0.922	0.98
Random Forest	Random Forest Regressor	0.942	0.76
Word2Vec	Linear Regressor	0.972	0.06

The results indicate strong predictive accuracy for both K-Means and Random Forest operators, particularly in predicting memory usage, with R^2 scores of 0.98 and 0.76, respectively. The Gradient Boosting Regressor (GBR) showed the highest performance for K-Means, achieving R^2 values of 0.922 for total time and 0.98 for memory usage. For the Random Forest operator, the Random Forest Regressor (RFR) performed best with R^2 scores of 0.942 for total time and 0.76 for memory usage.

On the other hand, the Word2Vec operator exhibited different behavior, with a Linear Regressor performing best in predicting total execution time ($R^2 = 0.972$), but showing poor accuracy in predicting memory usage ($R^2 = 0.06$). This discrepancy suggests that the memory consumption patterns for Word2Vec are more complex and may require more sophisticated modeling techniques, possibly due to the large-scale text data and the algorithm’s complexity.

These results emphasize the importance of selecting appropriate models for different operators and metrics. For structured data, such as in K-Means and Random Forest, ensemble models like Gradient Boosting and Random Forest Regressor performed well. In contrast, the simpler Linear Regressor was suitable for predicting execution time in the Word2Vec operator, but inadequate for memory usage prediction. The poor performance in memory prediction for Word2Vec suggests the need for additional research into more advanced techniques, possibly by incorporating more features or preprocessing steps to better capture the memory profiles of text-based algorithms.

Overall, this study demonstrates how predictive modeling can enhance our understanding of the performance of Apache Spark’s MLlib operators. By using machine learning models, we can anticipate execution time and memory usage more effectively, which is crucial for optimizing the performance of distributed systems. Future work could focus on improving the predictability of memory usage for complex operators like Word2Vec and exploring new machine learning methods to further refine the capabilities of Spark’s MLlib in large-scale data processing.

Real-World Applications and Limitations

- **Real-World Scenarios: Applications:** The predictive models in this study have direct application in several industrial settings. To further extend this application, the models perform the optimization of resource allocation themselves for quality control processes, including defect detection via the application of machine learning, improving efficiency and eliminating downtime while increasing production throughput in the manufacturing industry, for example. For healthcare analytics, predictive models

can be used to plan costs and generate timeliness of the results in priorities such as medical image processing and patient outcome predictions.

These models can help in logistics and supply chain management where real time analytics workflows such as route planning and inventory control incur delays in processing large datasets and can cause financial inefficiencies. In such dynamic environments, it provides the ability to carry out proactive scheduling and resource management, i.e., the ability to forecast and execute time and memory usage.

- **Challenges and limitations:** There are a number of limitations. During training, the predictive models have computational overhead, and at a sufficient scale, this overhead could rigorously outweigh the resource savings provided during execution. Moreover, implementing these models within existing Apache Spark workflows with architectural modifications is needed. A limitation of this work is dependency on hyperparameter tuning, due to which it may be challenging to utilize in a resource-constrained or expertise-scarce environment.
- **Future Directions:** To address these limitations, future research could be based on using hybrid modeling techniques to make a predictive task scalable and robust. Furthermore, by embedding automated hyperparameter optimization frameworks within the predictive modeling process, we can simplify the deployment of these tools for broader industrial use.

6. Conclusions

This study demonstrated the feasibility of using predictive models to estimate the performance of Apache Spark's MLlib operators by focusing on two key metrics: It saves complete info regarding total execution time as well as reducing the consumption of memory. To provide a broad set for evaluation, we ran experiments across each aspect of dataset size, number of features, and sample counts, for three major machine learning operators: K-Means, Random Forest, and Word2Vec. The results show that even though both execution time and memory can be accurately modeled using strong predictive models such as Gradient Boosting and Random Forest Regressor, it is harder to fit some structured data algorithms such as K-Means and Random Forest. The R^2 scores for these models are very high, which suggests that relationships between dataset characteristics and operator performance can be modeled using nonlinear techniques.

Nevertheless, there are also some limitations highlighted in the results, in particular for unstructured data algorithms such as Word2Vec. Although Linear Regression mostly accounted for the total execution time, the low R^2 of memory usage describes that simple linear models cannot be used to capture the memory usage patterns of more complex operators. This finding highlights the importance of developing more sophisticated models that can handle nonlinear behavior created by algorithms that process textual data and manipulate unstructured inputs; a new area of study may be needed.

While this study provided valuable insights into predictive modeling for structured algorithms like K-Means and Random Forest, it also highlighted significant challenges in predicting memory usage for unstructured data algorithms, such as Word2Vec. Addressing these challenges requires the development of sophisticated nonlinear models and enhanced preprocessing techniques tailored to the complexities of unstructured data processing. These refinements are essential for improving predictive accuracy and robustness across diverse machine learning operators in distributed frameworks.

The key contribution of this work is that predictive models are a viable tool for optimizing resource allocation in distributed machine learning environments. In large-scale distributed systems such as Apache Spark, job scheduling and resource management are critical, and we need accurate predictions of execution time and memory usage. Using

predictive models, data engineers not only can reduce the chance of data processes running into memory exhaustion or bottlenecks, but they can better know what resources they might need in advance.

This study offered valuable insights about K-Means and Random Forest, but algorithms like Word2Vec still present a problem. Further research into nonlinear models and feature engineering as well as advanced data preprocessing methods is needed to address the need for more refined memory usage predictions. In order to improve the overall accuracy and robustness of predictive models of any type of machine learning operator in distributed frameworks, these refinements will be critical.

7. Future Work

Several potential improvements can be built upon the findings of this study. The first is that a clear opportunity exists to see if more advanced machine learning models than what we tested can predict memory usage better, such as Word2Vec. Additional techniques such as deep learning, neural networks, or ensemble may be better at capturing the nonlinearity caused by such unstructured data processing.

Further research could also be performed to improve the data preprocessing and feature extraction techniques used in this study. If we make the model harder—using word frequency distributions or text complexity metrics for Word2Vec, for instance—perhaps we can better predict memory usage. Other techniques for dimensionality reduction, or feature selection, would also be candidates to help improve the generalization of predictive models.

Future work may also evaluate the scaling of developed models over larger, more heterogeneous datasets. Running these predictive models in different distributed environments and measuring in what ways they are robust and applicable to real-world problems would give valuable insights. Further, the direct incorporation of such predictive models within the Apache Spark framework may obtain substantial performance improvements. By allowing Spark to know in advance how long it will take to execute and how much memory it needs based on dataset characteristics, scheduling and resource management could be dramatically improved, eliminating computational overhead and reducing the waste of machine time and resources. Below are some future recommendations.

- **Deep Learning for Word2Vec Memory Prediction:** Future research should explore advanced machine learning models, such as Recurrent Neural Networks (RNNs) or Transformers, to improve memory usage forecasting for algorithms handling unstructured data. These models are designed to capture complex dependencies and nonlinear relationships inherent in text-based workloads.
- **Feature Engineering Enhancements:** Introducing advanced features like word frequency distributions, text complexity metrics, and hierarchical data representations could enhance the predictive accuracy for memory usage. Dimensionality reduction and feature selection techniques should also be investigated to improve generalization across datasets.
- **Scalability Analysis:** Testing the developed models on larger, more heterogeneous datasets across various distributed environments can provide valuable insights into their scalability and adaptability to real-world big data scenarios.
- **Integration into Apache Spark:** Embedding predictive models directly into Apache Spark could optimize job scheduling and resource management. By enabling the framework to anticipate computational demands based on dataset characteristics, significant reductions in execution overhead and resource wastage could be achieved.
- **Extension to Other Operators:** Extending the predictive modeling approach to other Spark MLlib operators or alternative distributed frameworks would generalize the findings, offering broader applicability for resource-efficient big data processing.

Finally future investigations may explore how other operators in Spark MLlib or other distributed frameworks can apply predictive modeling. Extending the utility of predictive modeling would further enhance the development of resource-efficient big data processing systems by investigating how general these models can be applied to other machine learning tasks, like classification or deep learning-based operators.

Author Contributions: L.T., A.K. and G.A.K. conceived of the idea, designed and constructed the experiments, drafted the initial manuscript, and revised the final manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Assali, T.; Ayoub, Z.T.; Ouni, S. Multivariate LSTM for Execution Time Prediction in HPC for Distributed Deep Learning Training. In Proceedings of the 2024 IEEE 27th International Symposium on Real-Time Distributed Computing (ISORC), Tunis, Tunisia, 22–25 May 2024; pp. 1–5. [\[CrossRef\]](#)
2. Salman, S.M.; Dao, V.L.; Papadopoulos, A.V.; Mubeen, S.; Nolte, T. Scheduling Firm Real-time Applications on the Edge with Single-bit Execution Time Prediction. In Proceedings of the 2023 IEEE 26th International Symposium on Real-Time Distributed Computing (ISORC), Nashville, TN, USA, 23–25 May 2023; pp. 207–213. [\[CrossRef\]](#)
3. Chen, R. Research on the Performance of Collaborative Filtering Algorithms in Library Book Recommendation Systems: Optimization of the Spark ALS Model. In Proceedings of the 2024 International Conference on Integrated Circuits and Communication Systems (ICICACS), Raichur, India, 23–24 February 2024; pp. 1–6. [\[CrossRef\]](#)
4. Han, M. Research on optimization of K-means Algorithm Based on Spark. In Proceedings of the 2023 IEEE 6th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), Chongqing, China, 24–26 February 2023; Volume 6, pp. 1829–1836. [\[CrossRef\]](#)
5. Pham, T.P.; Durillo, J.J.; Fahringer, T. Predicting workflow task execution time in the cloud using a two-stage machine learning approach. *IEEE Trans. Cloud Comput.* **2017**, *8*, 256–268. [\[CrossRef\]](#)
6. Balis, B.; Lelek, T.; Boder, J.; Grabowski, M.; Grigoras, C. Improving prediction of computational job execution times with machine learning. *Concurr. Comput. Pract. Exp.* **2024**, *36*, e7905. [\[CrossRef\]](#)
7. Schizas, N.; Karras, A.; Karras, C.; Sioutas, S. TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review. *Future Internet* **2022**, *14*, 363. [\[CrossRef\]](#)
8. Karras, A.; Giannaros, A.; Theodorakopoulos, L.; Krimpas, G.A.; Kalogeratos, G.; Karras, C.; Sioutas, S. FLIBD: A federated learning-based IoT big data management approach for privacy-preserving over Apache Spark with FATE. *Electronics* **2023**, *12*, 4633. [\[CrossRef\]](#)
9. Karras, A.; Karras, C.; Giotopoulos, K.C.; Tsolis, D.; Oikonomou, K.; Sioutas, S. Federated Edge Intelligence and Edge Caching Mechanisms. *Information* **2023**, *14*, 414. [\[CrossRef\]](#)
10. Sewal, P.; Singh, H. A Machine Learning Approach for Predicting Execution Statistics of Spark Application. In Proceedings of the 2022 Seventh International Conference on Parallel, Distributed and Grid Computing (PDGC), Solan, India, 25–27 November 2022; pp. 331–336. [\[CrossRef\]](#)
11. Ye, G.; Liu, W.; Wu, C.Q.; Shen, W.; Lyu, X. On Machine Learning-based Stage-aware Performance Prediction of Spark Applications. In Proceedings of the 2020 IEEE 39th International Performance Computing and Communications Conference (IPCCC), Austin, TX, USA, 6–8 November 2020; pp. 1–8. [\[CrossRef\]](#)
12. Ataie, E.; Evangelinou, A.; Gianniti, E.; Ardagna, D. A hybrid machine learning approach for performance modeling of cloud-based big data applications. *Comput. J.* **2022**, *65*, 3123–3140. [\[CrossRef\]](#)
13. Gulino, A.; Canakoglu, A.; Ceri, S.; Ardagna, D. Performance Prediction for Data-driven Workflows on Apache Spark. In Proceedings of the 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), Nice, France, 17–19 November 2020; pp. 1–8. [\[CrossRef\]](#)
14. Tsai, L.; Franke, H.; Li, C.S.; Liao, W. Learning-Based Memory Allocation Optimization for Delay-Sensitive Big Data Processing. *IEEE Trans. Parallel Distrib. Syst.* **2018**, *29*, 1332–1341. [\[CrossRef\]](#)

15. Gárate-Escamilla, A.K.; El Hassani, A.H.; Andres, E. Big data execution time based on Spark Machine Learning Libraries. In Proceedings of the 2019 3rd International Conference on Cloud and Big Data Computing, Oxford, UK, 28–30 August 2019; pp. 78–83.
16. Wang, G.; Xu, J.; He, B. A Novel Method for Tuning Configuration Parameters of Spark Based on Machine Learning. In Proceedings of the 2016 IEEE 18th International Conference on High Performance Computing and Communications; IEEE 14th International Conference on Smart City; IEEE 2nd International Conference on Data Science and Systems (HPCC/SmartCity/DSS), Sydney, NSW, Australia, 12–14 December 2016; pp. 586–593. [\[CrossRef\]](#)
17. Lu, X.; Shankar, D.; Gugnani, S.; Panda, D.K. High-performance design of apache spark with RDMA and its benefits on various workloads. In Proceedings of the 2016 IEEE International Conference on Big Data (Big Data), Washington, DC, USA, 5–8 December 2016; pp. 253–262. [\[CrossRef\]](#)
18. Manzi, D.; Tompkins, D. Exploring GPU Acceleration of Apache Spark. In Proceedings of the 2016 IEEE International Conference on Cloud Engineering (IC2E), Berlin, Germany, 4–8 April 2016; pp. 222–223. [\[CrossRef\]](#)
19. Öztürk, M.M. MFRLMO: Model-free reinforcement learning for multi-objective optimization of apache spark. *EAI Endorsed Trans. Scalable Inf. Syst.* **2024**, *11*, 1–15. [\[CrossRef\]](#)
20. Ishizaki, K. Analyzing and optimizing java code generation for apache spark query plan. In Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering, Mumbai, India, 7–11 April 2019; pp. 91–102.
21. Giannaros, A.; Karras, A.; Theodorakopoulos, L.; Karras, C.; Kranias, P.; Schizas, N.; Kalogeratos, G.; Tsolis, D. Autonomous vehicles: Sophisticated attacks, safety issues, challenges, open topics, blockchain, and future directions. *J. Cybersecur. Priv.* **2023**, *3*, 493–543. [\[CrossRef\]](#)
22. Theodorakopoulos, L.; Karras, A.; Theodoropoulou, A.; Kampiotis, G. Benchmarking Big Data Systems: Performance and Decision-Making Implications in Emerging Technologies. *Technologies* **2024**, *12*, 217. [\[CrossRef\]](#)
23. Karras, A.; Giannaros, A.; Karras, C.; Theodorakopoulos, L.; Mammassis, C.S.; Krimpas, G.A.; Sioutas, S. TinyML algorithms for Big Data Management in large-scale IoT systems. *Future Internet* **2024**, *16*, 42. [\[CrossRef\]](#)
24. Dong, C.; Akram, A.; Andersson, D.; Arnäs, P.O.; Stefansson, G. The impact of emerging and disruptive technologies on freight transportation in the digital era: Current state and future trends. *Int. J. Logist. Manag.* **2021**, *32*, 386–412. [\[CrossRef\]](#)
25. Ohlhorst, F.J. *Big Data Analytics: Turning Big Data into Big Money*; John Wiley & Sons: Hoboken, NJ, USA, 2012; Volume 65.
26. Vummadi, J.; Hajarath, K. Integration of Emerging Technologies AI and ML into Strategic Supply Chain Planning Processes to Enhance Decision-Making and Agility. *Int. J. Supply Chain. Manag.* **2024**, *9*, 77–87. [\[CrossRef\]](#)
27. Sun, Z. Intelligent big data analytics: A managerial perspective. In *Managerial Perspectives on Intelligent Big Data Analytics*; IGI Global: Hershey, PA, USA 2019; pp. 1–19.
28. Pouyanfar, S.; Yang, Y.; Chen, S.C.; Shyu, M.L.; Iyengar, S. Multimedia big data analytics: A survey. *ACM Comput. Surv. (CSUR)* **2018**, *51*, 1–34. [\[CrossRef\]](#)
29. Sterling, M. Situated big data and big data analytics for healthcare. In Proceedings of the 2017 IEEE Global Humanitarian Technology Conference (GHTC), San Jose, CA, USA, 19–22 October 2017; p. 1. [\[CrossRef\]](#)
30. Ularu, E.G.; Puican, F.C.; Apostu, A.; Velicanu, M. Perspectives on big data and big data analytics. *Database Syst. J.* **2012**, *3*, 3–14.
31. Crowder, J.A.; Carbone, J.; Friess, S.; Crowder, J.A.; Carbone, J.; Friess, S. Data analytics: The big data analytics process (bdap) architecture. In *Artificial Psychology: Psychological Modeling and Testing of AI Systems*; Springer: Cham, Switzerland, 2020; pp. 149–159.
32. Padilha, B.; Schwerz, A.L.; Roberto, R.L. WED-SQL: A Relational Framework for Design and Implementation of Process-Aware Information Systems. In Proceedings of the 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW), Atlanta, GA, USA, 5–8 June 2017; pp. 364–369. [\[CrossRef\]](#)
33. Udoh, I.S.; Kotonya, G. Developing IoT applications: Challenges and frameworks. *IET Cyber-Phys. Syst. Theory Appl.* **2018**, *3*, 65–72. [\[CrossRef\]](#)
34. Horii, S. Improved computation-communication trade-off for coded distributed computing using linear dependence of intermediate values. In Proceedings of the 2020 IEEE International Symposium on Information Theory (ISIT), Los Angeles, CA, USA, 21–26 June 2020; pp. 179–184.
35. Yan, Q.; Yang, S.; Wigger, M. Storage-Computation-Communication Tradeoff in Distributed Computing: Fundamental Limits and Complexity. *IEEE Trans. Inf. Theory* **2022**, *68*, 5496–5512. [\[CrossRef\]](#)
36. Jangda, A.; Huang, J.; Liu, G.; Sabet, A.H.N.; Maleki, S.; Miao, Y.; Musuvathi, M.; Mytkowicz, T.; Saarikivi, O. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, 28 February–4 March 2022; pp. 402–416.
37. Hu, H.; Jiang, C.; Zhong, Y.; Peng, Y.; Wu, C.; Zhu, Y.; Lin, H.; Guo, C. dPRO: A Generic Profiling and Optimization System for Expediting Distributed DNN Training. *arXiv* **2022**, arXiv:2205.02473.

38. Cheng, D.; Wang, Y.; Dai, D. Dynamic resource provisioning for iterative workloads on Apache Spark. *IEEE Trans. Cloud Comput.* **2021**, *11*, 639–652. [\[CrossRef\]](#)
39. Kordelas, A.; Spyrou, T.; Voulgaris, S.; Megalooikonomou, V.; Deligiannis, N. KORDI: A Framework for Real-Time Performance and Cost Optimization of Apache Spark Streaming. In Proceedings of the 2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Raleigh, NC, USA, 23–25 April 2023; pp. 337–339.
40. Cheng, G.; Ying, S.; Wang, B. Tuning configuration of apache spark on public clouds by combining multi-objective optimization and performance prediction model. *J. Syst. Softw.* **2021**, *180*, 111028. [\[CrossRef\]](#)
41. Geng, J.; Li, D.; Cheng, Y.; Wang, S.; Li, J. HiPS: Hierarchical parameter synchronization in large-scale distributed machine learning. In Proceedings of the 2018 Workshop on Network Meets AI & ML, Budapest, Hungary, 24 August 2018; pp. 1–7.
42. Nascimento, J.P.B.; Capanema, D.O.; Pereira, A.C.M. Assessing and improving the performance and scalability of an iterative algorithm for Hadoop. In Proceedings of the 2017 Computing Conference, London, UK, 18–20 July 2017; pp. 1069–1076. [\[CrossRef\]](#)
43. Sahith, C.S.K.; Muppidi, S.; Merugula, S. Apache Spark Big data Analysis, Performance Tuning, and Spark Application Optimization. In Proceedings of the 2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT), Bengaluru, India, 20–21 October 2023; pp. 1–8.
44. Ousterhout, K. Architecting for Performance Clarity in Data Analytics Frameworks. Ph.D. Thesis, UC Berkeley, Berkeley, CA, USA, 2017.
45. Dubey, R.; Gunasekaran, A.; Childe, S.J.; Blome, C.; Papadopoulos, T. Big data and predictive analytics and manufacturing performance: Integrating institutional theory, resource-based view and big data culture. *Br. J. Manag.* **2019**, *30*, 341–361. [\[CrossRef\]](#)
46. Gupta, Y.K.; Kumari, S. Performance Evaluation of Distributed Machine Learning for Cardiovascular Disease Prediction in Spark. In Proceedings of the 2021 5th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 3–5 June 2021; pp. 1506–1512.
47. Assefi, M.; Behraves, E.; Liu, G.; Tafti, A.P. Big data machine learning using apache spark MLlib. In Proceedings of the 2017 IEEE International Conference on Big Data (Big Data), Boston, MA, USA, 11–14 December 2017; pp. 3492–3498. [\[CrossRef\]](#)
48. Atefinia, R.; Ahmadi, M. Performance evaluation of Apache Spark MLlib algorithms on an intrusion detection dataset. *arXiv* **2022**, arXiv:2212.05269.
49. Karras, A.; Karras, C.; Bompotas, A.; Bouras, P.; Theodorakopoulos, L.; Sioutas, S. SparkReact: A Novel and User-friendly Graphical Interface for the Apache Spark MLlib Library. In Proceedings of the 26th Pan-Hellenic Conference on Informatics, Athens, Greece, 25–27 November 2022; pp. 230–239.
50. Qadri, A.M.; Raza, A.; Munir, K.; Almutairi, M.S. Effective Feature Engineering Technique for Heart Disease Prediction with Machine Learning. *IEEE Access* **2023**, *11*, 56214–56224. [\[CrossRef\]](#)
51. Azeroual, O.; Nikiforova, A. Apache spark and mllib-based intrusion detection system or how the big data technologies can secure the data. *Information* **2022**, *13*, 58. [\[CrossRef\]](#)
52. Esmaeilzadeh, A.; Heidari, M.; Abdolazimi, R.; Hajibabae, P.; Malekzadeh, M. Efficient Large Scale NLP Feature Engineering with Apache Spark. In Proceedings of the 2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 26–29 January 2022; pp. 0274–0280. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.